

VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5

Student Name	Registration Number
PRIYA	24MEI10017
Slot	Date of Submission
F11	31.03.26

Part A — Origin and Philosophy (Units 1 & 2)

1. The problem GIT was created to solve

Before Git existed, software developers had a big problem. When many people work on the same code at the same time, things get messy. Someone might overwrite another person's work. It becomes very hard to track who changed what and when.

In the early 2000s, the Linux kernel team was using a version control tool called BitKeeper. BitKeeper was a proprietary software — meaning it was owned by a company and not free. The Linux community was allowed to use it for free, but only under certain conditions.

In 2005, a developer named **Andrew Tridgell** tried to reverse-engineer BitKeeper to understand how it worked. The company that owned BitKeeper did not like this and cancelled the free license for the Linux community. This left Linus Torvalds — the creator of Linux — without a tool to manage his own project.

Linus was frustrated. He looked at other version control tools available at the time but none of them met his needs. They were either too slow, too centralised, or not free enough. So he did what any great open source developer would do — he built his own tool in just 10 days.

He called it **Git**. The name is British slang for a stupid or unpleasant person — Linus joked that he always names his projects after himself.

Git was designed with three main goals:

- **Speed** — it had to be fast enough to handle the Linux kernel which is a massive codebase
- **Distributed** — every developer gets a full copy of the entire history, not just the latest version
- **Free and Open** — licensed under GPL v2 so no company could ever take it away again

Git was first released on **7 April 2005** and within months it was already being used to manage the Linux kernel itself.

A2. The license — what it actually says

Git is licensed under the **GNU General Public License version 2 (GPL v2)**. This is one of the most well known open source licenses in the world.

The Four Freedoms of Free Software

Richard Stallman of the GNU Project defined four freedoms that every free software must give its users:

Freedom 0 :You can **run** the software for any purpose

Freedom 1 :You can **study** how the software works and change it

Freedom 2 :You can **share** copies of the software with others

Freedom 3 :You can **improve** the software and share your improvements

Git's GPL v2 license grants **all four freedoms**. Anyone in the world can download Git, read its source code, modify it, and share their version freely.

Can a Company Modify Git and Sell It Without Sharing?

No. This is the most important feature of the GPL license. It has a concept called **copyleft** which means if you modify GPL software and distribute it, you **must** release your modified version under the same GPL license. You cannot take Git, change it, and sell it as a closed proprietary product. You must share your changes with the world.

This is very different from a license like MIT, where a company can take the code, modify it, and keep their changes private.

Difference Between GPL and MIT

	GPL v2	MIT
Can use freely	Yes	Yes
Must share changes	Yes	No
Can make it proprietary	No	Yes
Best for	Community protection	Maximum freedom

If I were building something I wanted to keep open forever and prevent companies from making proprietary versions of it, I would choose **GPL**. If I just wanted people to use my code freely without any restrictions, I would choose **MIT**.

Free as in Free Beer vs Free as in Freedom

These are two very different meanings of the word "free."

Free as in free beer means it costs nothing. You can have it at no price but you cannot necessarily change it or share it. For example, many proprietary apps are free to download but you cannot see or modify their code.

Free as in freedom means you have the right to do whatever you want with it — use it, study it, change it, share it. Git is free in this second sense. It costs nothing AND you have full freedom over it. That is what makes it truly open source.

A3. The ethics of open source

Should All Software Be Open Source?

When I first started this course I thought the answer to this was obvious — of course all software should be open source. Why would anyone want to hide code? But after thinking about it properly I realized it is not that simple.

The argument for making all software open source:

When code is open anyone can look at it and find problems. Security researchers, students, governments, hospitals — anyone can check if the software is safe. This is especially important for software that runs critical systems like banking, healthcare, or elections. If the code is hidden, how do we know it is not doing something harmful in the background?

Open source also makes powerful tools available to people who cannot afford expensive software. A student in a small town can use the same tools as a developer at Google — because Linux, Python, Git, and Firefox are all free. This levels the playing field in a way that proprietary software never could.

There is also the question of knowledge. Software is not just a product — it is knowledge written in code. Keeping knowledge locked away slows down the entire world. Every time a developer reads open source code they learn something. Every time someone fixes a bug in a public project, thousands of other people benefit from that fix automatically.

The argument against making all software open source:

Building software is expensive. It takes months or years of work from highly skilled people who need to be paid. If a company cannot protect its code and sell it, how does it make money to pay those developers? Some level of proprietary software is what funds innovation in the first place.

There are also genuine security concerns. If the source code of a banking system or a military system is publicly available, it becomes easier for attackers to find and exploit weaknesses. Sometimes keeping code private is actually safer.

And not all open source projects stay well maintained. When a key developer leaves and nobody takes over, the project gets abandoned. Proprietary software at least has a company with a financial reason to keep it working.

My conclusion:

I think the answer is somewhere in the middle. Core infrastructure that everyone depends on — operating systems, programming languages, web servers — should be open source because they are like public roads. But companies should be allowed to build proprietary products on top of that foundation. The important thing is that the foundation stays free.

Is It Ethical for Companies to Profit From Free Community Labor?

This question made me genuinely uncomfortable because I had never thought about it before. When I use GitHub or Google Search I never thought about the fact that both of those services run on Linux — software that was built by thousands of volunteers and individual developers who were never paid by Google.

Google, Meta, Amazon, and Microsoft have all built billion dollar businesses on top of Linux and other open source tools. They did not compensate most of the people who built those tools.

At first this seems completely unfair. Imagine spending years building something for free and then watching a trillion dollar company use it to make billions of rupees without even saying thank you.

But then I thought about it more carefully. Many of these companies do give back. Google has contributed millions of lines of code back to Linux and Android. Microsoft now contributes heavily to open source and even made VS Code free. Red Hat's entire business model is built around Linux and they employ many of the core Linux developers full time.

The problem is when a company **only takes and never gives back**.

There is no law against this but I personally think it is unethical. The open source community works on trust — the idea that we all benefit and we all contribute what we can. A company that only extracts value without contributing anything is breaking that trust even if it is not breaking any rules.

The most honest answer I can give is this — it is ethical to build a business on open source as long as you contribute something back. It is unethical to treat open source as a free buffet with no responsibility to the people who cooked the food.

Standing on the Shoulders of Giants — Does Open Source Enable or Undermine Innovation?

The phrase "standing on the shoulders of giants" means that every new achievement is only possible because of the work that came before it. Nobody invents something completely from scratch. Every idea builds on earlier ideas.

In software this is more true than in almost any other field. When I

write a Python script I am standing on the shoulders of the people who built Python. Python itself stands on the shoulders of C. C stands on earlier assembly languages. Every layer of technology was built by people who came before.

Git is a perfect example of this. Before Git there was SVN. Before SVN there was CVS. Linus Torvalds studied all of these systems, understood what they did wrong, and built something better. He could only do this because those earlier systems were available to study. If CVS and SVN had been completely proprietary with no public documentation, Git might never have existed.

Does open source enable innovation?

Yes, absolutely. When code is open anyone can learn from it, improve it, and build something new on top of it. The entire modern internet — web servers, databases, programming languages, operating systems — is built on open source foundations. This did not happen by accident. It happened because people shared their work freely and others built on it.

Does open source undermine innovation?

Some people argue that if you cannot protect your code, there is no reward for innovation. Why spend three years building something if anyone can just copy it for free?

I think this argument misunderstands how open source works. The people who build open source software are not doing it for nothing. They get reputation, job opportunities, the satisfaction of building something used by millions, and often financial support from companies or donations. Linus Torvalds did not build Git because he wanted to get rich. He built it because he needed it and he shared it because that is what the open source philosophy is about.

In my opinion open source does not undermine innovation — it **accelerates** it. Because when one person solves a problem and shares the solution, a thousand other people do not have to solve the same problem again. They can spend their energy on the next problem. That is how progress actually works.

Part B — Linux Footprint of Git (Unit 2)

To understand how Git functions within a Linux environment, I installed and explored it on an Ubuntu-based system. This helped me observe how open-source software integrates with the operating system in a practical way.

Installation Method

Git is available through the default package manager in Linux. On my system, I installed it using the APT package manager with the command `sudo apt install git`.

The fact that Git is available directly through the official repositories shows that it is widely trusted and actively maintained by the open-source community. It also makes installation simple, without requiring manual compilation.

Key Directories

After installation, Git is not located in a single place but is distributed across standard Linux directories.

The main executable file is located in `/usr/bin/git`, which is used to run all Git commands. Configuration files are stored in two main locations: `/etc/gitconfig` for system-wide settings and `~/.gitconfig` for user-specific configurations. Additionally, supporting files such as templates and documentation are stored in `/usr/share/git/`.

This structure follows the standard Linux filesystem hierarchy, which keeps binaries, configuration, and resources organized separately.

User and Group

Unlike some software that runs as a background service, Git operates under the currently logged-in user. It does not require a dedicated system user or group.

This means that all Git operations are performed with the same permissions as the user running the command. From a security perspective, this is beneficial because Git cannot access or modify files beyond the user's permission level.

Service Management

Git does not run as a service or daemon in the background. Instead, it is a command-line tool that is executed only when needed.

Commands such as `git init`, `git clone`, and `git status` are used interactively by the user. Since there is no continuous background process, Git is lightweight and does not consume system resources unnecessarily.

Update Model

Git is updated through the Linux package management system. When system

updates are performed, Git is also updated along with other installed packages.

These updates include bug fixes, security patches, and performance improvements contributed by the open-source community. This reflects how open-source software evolves continuously through collaborative effort.

Overall Observation

From my exploration, I found that Git integrates very efficiently with Linux. It does not require complex setup, runs with minimal system resources, and follows standard directory structures.

This simplicity and efficiency reflect the core philosophy of open source — building tools that are transparent, reliable, and accessible to everyone.

Part C — The FOSS Ecosystem (Units 3 & 4)

No open-source project exists independently, and Git is a perfect example of how software grows within a larger ecosystem of tools, communities, and collaborations.

Dependencies (Libraries, Compilers, Runtimes)

Git is written primarily in the C programming language, so it depends on standard system libraries available on Linux. It is compiled using tools like **GCC**, which is itself an open-source compiler.

It also makes use of:

- Standard C libraries for system-level operations
- Shell tools like Bash for scripting support
- Libraries such as OpenSSL for secure communication (in some configurations)

This shows that even a powerful tool like Git is built on top of other open-source components, forming a layered ecosystem.

What Git Has Enabled or Inspired

Git has not only solved version control problems but has also enabled an entire ecosystem of platforms and tools.

The most prominent example is **GitHub**, which allows developers to host repositories, collaborate, and contribute to open-source projects globally. Similarly, **GitLab** provides integrated DevOps features on top of Git.

Git has also inspired:

- Fork-based collaboration models
- Continuous Integration/Continuous Deployment (CI/CD) pipelines
- Tools like GitKraken and SourceTree for graphical interfaces

In a way, Git has become the foundation of modern software collaboration.

Relation to LAMP Stack / Web

Git is not directly a part of the traditional LAMP stack (Linux, Apache, MySQL, PHP), but it plays an important supporting role in web development.

Developers use Git to:

- Manage source code for web applications
- Track changes in backend (PHP, Node.js, etc.) and frontend code
- Collaborate on web projects efficiently

For example, a web application running on a LAMP stack is often developed and maintained using Git for version control. This makes Git an essential tool in the development lifecycle, even if it is not part of the runtime stack.

Community and Governance

Git has a strong and active open-source community. Although it was created by **Linus Torvalds**, its development is now maintained by a group of contributors.

The community interacts through:

- Mailing lists (especially the Git mailing list for discussions and patches)
- Online forums and documentation
- Platforms like GitHub where issues and pull requests are managed

Unlike some projects controlled by a single company, Git follows a **community-driven development model**. Contributions are reviewed and accepted based on merit, ensuring quality and transparency.

Overall Understanding

From this ecosystem analysis, I understood that Git is not just a standalone tool. It is deeply connected to other open-source technologies and has itself become a foundation for modern development practices.

It both depends on open-source tools and enables new ones, creating a continuous cycle of innovation within the FOSS ecosystem.

Part D — Open Source vs Proprietary

Dimension	Open Source	Proprietary Alternative(Bitkeeper)
<i>Cost</i>	Completely free to use, modify, and distribute	Requires paid licenses, especially for enterprise use
<i>Security</i>	Source code is open, so anyone can audit and identify vulnerabilities	Code is closed; only the company can audit and fix issues
<i>Support and reliability</i>	Community-driven support; large user base and documentation	large user base and documentation Dedicated professional support, often faster and structured
<i>Freedom to modify</i>	Full freedom to modify and redistribute under GPL license	No access to source code; modifications are restricted

<i>Community vs corporate control</i>	Community-driven development with contributions from developers worldwide	Controlled by a single company with business priorities
<i>Your overall verdict</i>	Flexible, transparent, and ideal for collaboration	Structured and supported, but limited in freedom

Conclusion

After comparing Git with proprietary alternatives, I believe Git is more suitable for most real-world use cases, especially in collaborative and open development environments. Its distributed nature, zero cost, and transparency make it highly accessible and reliable. While proprietary tools may offer better official support and structured environments, they lack the flexibility and openness that Git provides.

Personally, I would recommend Git for real-world deployment because it is already widely adopted, well-documented, and continuously improved by a global community. I would also consider contributing to it in the future, even if in a small way, such as reporting issues or improving documentation. Being part of an open-source ecosystem like Git feels more meaningful because it allows learning, sharing, and collaboration at a global scale

1. Shell Script Tasks

Script 1 — System Identity Report

This script displays important system information such as kernel version, current user, system uptime, and date in a formatted output. It collects this data using commands like `uname`, `whoami`, and `uptime`. The script uses variables to store values, command substitution (`$()`) to execute commands, and `echo` for formatted output. It demonstrates basic shell scripting concepts like variables and output formatting.

INPUT:

```
scripts > $ script1.sh
1  #!/bin/bash
2  # Script 1: System Identity Report
3  # Created by Priya (24MEI10017)
4  # Course: Open Source Software
5
6  STUDENT_NAME="Priya"
7  SOFTWARE_CHOICE="Git"
8  LICENSE="GNU General Public License version 2 (GPL v2)"
9
10 KERNEL=$(uname -r)
11 USER_NAME=$(whoami)
12 HOME_DIR=$HOME
13 UPTIME=$(uptime -p)
14 DATE_TIME=$(date '+%d %B %Y, %H:%M:%S')
15 DISTRO=$(grep PRETTY_NAME /etc/os-release | cut -d= -f2 | tr -d ' "')
16
17 echo ""
18 echo " Hey there, $USER_NAME!"
19 echo " Welcome to the Open Source Audit Report"
20 echo ""
21 echo " Here's a quick snapshot of my system:"
22 echo ""
23
24 echo "=====
25 echo "          ABOUT THIS MACHINE"
26 echo "=====
27 echo ""
28
29 echo " Distribution : $DISTRO"
30 echo " Kernel Version : $KERNEL"
31 echo ""
```

```
32
33 echo " Logged-in User: $USER_NAME"
34 echo " Home Directory : $HOME_DIR"
35 echo ""
36
37 echo "=====
38 echo "          RIGHT NOW"
39 echo "=====
40 echo ""
41
42 echo " System Uptime : $UPTIME"
43 echo " (System seems to be running fine )"
44 echo " Date and Time : $DATE_TIME"
45 echo ""
46
47 echo "=====
48 echo "          OPEN SOURCE LICENSE"
49 echo "=====
50 echo ""
51
52 echo " Software Audited : $SOFTWARE_CHOICE"
53 echo " License : $LICENSE"
54 echo ""
55
56 echo " This means anyone can freely use, read,"
57 echo " modify and share this software."
58 echo " That's what makes open source powerful "
59 echo ""
60
61 echo "=====
62 echo " End of report."
63 echo ""
```

OUTPUT:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
@Priyasinghh28 → /workspaces/oss-audit-24MEI10017 (main) $ bash scripts/script1.sh
Here's a quick snapshot of my system:

=====
                        ABOUT THIS MACHINE
=====

Distribution   : Ubuntu 24.04.3 LTS
Kernel Version : 6.8.0-1044-azure

Logged-in User: codespace
Home Directory : /home/codespace

=====
                        RIGHT NOW
=====

System Uptime  : up 1 hour, 56 minutes
(System seems to be running fine )
Date and Time  : 29 March 2026, 16:21:46

=====
                        OPEN SOURCE LICENSE
=====

Software Audited : Git
License           : GNU General Public License version 2 (GPL v2)

This means anyone can freely use, read,
modify and share this software.
That's what makes open source powerful

=====
End of report.
```

Script 2 — FOSS Package Inspector

This script checks whether a specific package (Git) is installed on the system and displays its details if present. It uses an if-then-else statement to verify installation and commands like `dpkg` or `rpm` with `grep` to extract information. A case statement is used to print a short description of the package. This script demonstrates conditional statements, case selection, and command piping.

INPUT:

```

scripts > $ script2.sh
1  #!/bin/bash
2  # Script 2: FOSS Package Inspector
3  # Author: Priya | Reg No: 24MEI10017
4  # Course: Open Source Software
5  # Description: Checks if a package is installed and prints its philosophy
6
7  # --- Set the package name ---
8  PACKAGE="git"
9
10 echo ""
11 echo "=====
12 echo "          FOSS PACKAGE INSPECTOR"
13 echo "=====
14 echo ""
15 echo "  Checking if $PACKAGE is installed in  system..."
16 echo ""
17
18 # --- if-then-else: Check if package is installed ---
19 # dpkg -l is used instead of rpm -q because we are on Ubuntu
20 if dpkg -l "$PACKAGE" &>/dev/null; then
21     echo "  ✓ $PACKAGE is installed"
22     echo ""
23     # pipe with grep: filters only Version, License, Summary from package info
24     dpkg -s "$PACKAGE" | grep -E 'Version|Homepage|Description'
25     echo ""
26     echo "  Found at: $(which $PACKAGE)"
27 else
28     echo "  ✗ $PACKAGE is NOT installed."
29     echo "  Install it using: sudo apt install $PACKAGE"
30 fi
31
32 echo ""
33 echo "=====OPEN SOURCE PHILOSOPHY NOTE=====
34 echo ""
35
36 # --- case statement: prints philosophy note based on package name ---
37 case $PACKAGE in
38     git)
39         echo "  Git: created by Linus Torvalds when proprietary tools failed him." ;;
40     apache2|httpd)
41         echo "  Apache: the web server that built the open internet." ;;
42     mysql)
43         echo "  MySQL: open source at the heart of millions of apps." ;;
44     firefox)
45         echo "  Firefox: a nonprofit fighting to keep the web open." ;;
46     vlc)
47         echo "  VLC: built by students in Paris – plays anything for free." ;;
48     python3)
49         echo "  Python: a language shaped entirely by its community." ;;
50     *)
51         echo "  $PACKAGE: a proud member of the open source ecosystem." ;;
52 esac
53
54 echo ""
55

```

OUTPUT:

```
@Priyasinghh28 →/workspaces/oss-audit-24MEI10017 (main) $ nano scripts/script2.sh
@Priyasinghh28 →/workspaces/oss-audit-24MEI10017 (main) $ bash scripts/script2.sh

=====
FOSS PACKAGE INSPECTOR
=====

Checking if git is installed in system...

✓git is installed

Version: 1:2.43.0-1ubuntu7.3
Description: fast, scalable, distributed revision control system
Homepage: https://git-scm.com/

Found at: /usr/local/bin/git

=====OPEN SOURCE PHILOSOPHY NOTE=====

Git: created by Linus Torvalds when proprietary tools failed him.
```

Script 3 — Disk and Permission Auditor

This script analyzes multiple system directories and displays their size, permissions, and ownership. It uses a for loop to iterate through a list of directories and checks their existence using conditional statements. Commands like `ls -ld` and `du -sh` are used to gather information, along with tools like `awk` and `cut` for text processing. It demonstrates loops, conditions, and data extraction.

INPUT:

```

scripts > $ script3.sh
1  #!/bin/bash
2
3  # Script 3: Disk and Permission Auditor
4  # Created by Priya (24MEI10017)
5  # Course: Open Source Software
6  # This script checks directory size, permissions, and ownership
7
8  DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")
9
10 echo ""
11 echo " Hey $USER! Let me check some system directories..."
12 echo ""
13 echo "===== DIRECTORY AUDIT REPORT ====="
14 echo ""
15
16 for DIR in "${DIRS[@]}; do
17     if [ -d "$DIR" ]; then
18         PERMS=$(ls -ld "$DIR" | awk '{print $1, $3, $4}')
19         SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
20
21         echo " Directory : $DIR"
22         echo " Permission : $PERMS"
23         echo " Size : $SIZE"
24         echo " (This might take a second for bigger folders)"
25         echo "-----"
26     else
27         echo " Hmm, $DIR is not present on this system"
28         echo "-----"
29     fi
30 done
31
32 echo ""
33 echo "===== GIT CONFIG CHECK ====="
34 echo ""
35
36 GIT_CONFIG="$HOME/.config/git"
37
38 if [ -d "$GIT_CONFIG" ]; then
39     echo " ✓ Git config directory found at: $GIT_CONFIG"
40     echo " Git configured properly "
41     echo ""
42     ls -ld "$GIT_CONFIG"
43 else
44     echo " Git config folder not found at $GIT_CONFIG"
45     echo " check for ~/.gitconfig file instead..."
46     echo ""
47
48     if [ -f "$HOME/.gitconfig" ]; then
49         echo " ✓ Found ~/.gitconfig file"
50         ls -la "$HOME/.gitconfig"
51     else
52         echo " No git config found. Git might not be configured yet."
53     fi
54 fi
55
56 echo ""
57 echo "===== "
58 echo " Audit complete!"
59 echo "===== "
60 echo ""

```

OUTPUT:


```
@Priyasinghh28 →/workspaces/oss-audit-24MEI10017 (main) $ bash scripts/script3.sh
Hey codespace! Let me check some system directories...

===== DIRECTORY AUDIT REPORT =====

Directory : /etc
Permission : drwxr-xr-x root root
Size : 3.3M
(This might take a second for bigger folders)
-----
Directory : /var/log
Permission : drwxr-xr-x root root
Size : 824K
(This might take a second for bigger folders)
-----
Directory : /home
Permission : drwxr-xr-x root root
Size : 284M
(This might take a second for bigger folders)
-----
Directory : /usr/bin
Permission : drwxr-xr-x root root
Size : 534M
(This might take a second for bigger folders)
-----
Directory : /tmp
Permission : drwxr-xrwt+ root root
Size : 104K
(This might take a second for bigger folders)
-----

===== GIT CONFIG CHECK =====

Git config folder not found at /home/codespace/.config/git
check for ~/.gitconfig file instead...

@Priyasinghh28 →/workspaces/oss-audit-24MEI10017 (main) $ bash scripts/script3.sh
-----
Directory : /var/log
Permission : drwxr-xr-x root root
Size : 824K
(This might take a second for bigger folders)
-----
Directory : /home
Permission : drwxr-xr-x root root
Size : 284M
(This might take a second for bigger folders)
-----
Directory : /usr/bin
Permission : drwxr-xr-x root root
Size : 534M
(This might take a second for bigger folders)
-----
Directory : /tmp
Permission : drwxr-xrwt+ root root
Size : 104K
(This might take a second for bigger folders)
-----

===== GIT CONFIG CHECK =====

Git config folder not found at /home/codespace/.config/git
check for ~/.gitconfig file instead...

✓ Found ~/.gitconfig file
-rw-r--r-- 1 codespace codespace 126 Mar 28 17:56 /home/codespace/.gitconfig

=====
Audit complete!
=====
```

Script 4 — Log File Analyzer

This script reads a log file and counts how many times a specific keyword appears. It processes the file line by line using a while read loop and checks for the keyword using an if condition with grep. A counter variable is used to keep track of matches. It also accepts input through command-line arguments. This script demonstrates loops, conditional statements, file handling, and argument usage.

INPUT:

```
1  #!/bin/bash
2
3  # Script 4: Log File Analyzer
4  # Created by Priya (24MEI10017)
5  # Course: Open Source Software
6
7  LOGFILE=$1
8  KEYWORD=${2:-"error"}
9
10 COUNT=0
11
12 echo ""
13 echo " scanning for a log file "
14 echo ""
15 echo "===== LOG FILE ANALYZER ====="
16 echo ""
17
18 if [ ! -f "$LOGFILE" ]; then
19     echo " no valid file given , creating a sample log ."
20     echo ""
21
22     LOGFILE="/tmp/sample.log"
23     echo "INFO: System started successfully" > "$LOGFILE"
24     echo "ERROR: Disk read failed" >> "$LOGFILE"
25     echo "WARNING: Memory running low" >> "$LOGFILE"
26     echo "ERROR: Network connection timeout" >> "$LOGFILE"
27     echo "INFO: Attempting to reconnect" >> "$LOGFILE"
28     echo "ERROR: Failed to connect to server" >> "$LOGFILE"
29     echo "INFO: System recovery initiated" >> "$LOGFILE"
30
31     echo " Sample log created at: $LOGFILE"
32 fi
```

```

scripts > $ script4.sh
34 RETRY=0
35 while [ ! -s "$LOGFILE" ] && [ $RETRY -lt 3 ]; do
36     echo " File empty, retrying... (attempt $((RETRY+1)))"
37     RETRY=$((RETRY + 1))
38     sleep 1
39 done
40
41 echo ""
42 echo " Analyzing file: $LOGFILE"
43 echo " Searching for: '$KEYWORD'"
44 echo " (Case-insensitive search)"
45 echo ""
46 echo "-----"
47
48 while IFS= read -r LINE; do
49     if echo "$LINE" | grep -iq "$KEYWORD"; then
50         COUNT=$((COUNT + 1))
51     fi
52 done < "$LOGFILE"
53
54 echo ""
55 echo " found '$KEYWORD' $COUNT times in the log"
56 echo ""
57 echo "-----"
58 echo ""
59 echo " Last 5 matching lines:"
60 echo ""
61
62 if [ "$COUNT" -eq 0 ]; then
63     echo " No matching lines found."
64 else
65     grep -i "$KEYWORD" "$LOGFILE" | tail -5
66 fi
67
68 echo ""
69 echo "===== "
70 echo " Done analyzing "
71 echo "===== "
72 echo ""
73

```

OUTPUT:

```
• @Priyasinghh28 →/workspaces/oss-audit-24MEI10017 (main) $ nano scripts/script4.sh
• @Priyasinghh28 →/workspaces/oss-audit-24MEI10017 (main) $ bash scripts/script4.sh /var/log/syslog error

scanning for a log file

===== LOG FILE ANALYZER =====

no valid file given , creating a sample log .

Sample log created at: /tmp/sample.log

Analyzing file: /tmp/sample.log
Searching for: 'error'
(Case-insensitive search)

-----

found 'error' 3 times in the log

-----

Last 5 matching lines:

ERROR: Disk read failed
ERROR: Network connection timeout
ERROR: Failed to connect to server

=====
Done analyzing
=====
```

Script 5 — The Open Source Manifesto Generator

This script interacts with the user by asking questions and generating a personalized manifesto based on the responses. It uses the read command for input, variables to store data, and string concatenation to create output. The final result is saved to a file using output redirection. It also uses the date command. This script demonstrates user input, file handling, and string manipulation.

INPUT:

```

7  echo ""
8  echo "=====
9  echo "      OPEN SOURCE MANIFESTO GENERATOR"
10 echo "=====
11 echo ""
12 echo "  Answer the following questions "
13 echo "  and generate your open source manifesto."
14 echo ""
15
16 read -p "  1. Name one open-source tool you use every day: " TOOL
17 echo ""
18 read -p "  2. In one word, what does freedom mean to you? " FREEDOM
19 echo ""
20 read -p "  3. Name one thing you would build and share freely: " BUILD
21 echo ""
22
23 DATE=$(date '+%d %B %Y')
24 OUTPUT="manifesto_$(whoami).txt"
25
26 echo "  Alright, generating your manifesto..."
27 echo ""
28
29 echo "===== " > "$OUTPUT"
30 echo "      MY OPEN SOURCE MANIFESTO" >> "$OUTPUT"
31 echo "      Written on: $DATE" >> "$OUTPUT"
32 echo "===== " >> "$OUTPUT"
33 echo "" >> "$OUTPUT"
34 echo "  (This is based on your answers above)" >> "$OUTPUT"
35 echo "" >> "$OUTPUT"
36
37 echo "  I believe that technology should serve people" >> "$OUTPUT"
38 echo "  not the other way around." >> "$OUTPUT"
39 echo "" >> "$OUTPUT"
40 echo "  Every single day, I use $TOOL – a tool built" >> "$OUTPUT"
41 echo "  openly and shared freely by people who believed" >> "$OUTPUT"
42 echo "  that knowledge belongs to everyone." >> "$OUTPUT"
43 echo "" >> "$OUTPUT"
44 echo "  To me, freedom means $FREEDOM – and that matters." >> "$OUTPUT"
45 echo "  In software, that is the right to read, change," >> "$OUTPUT"
46 echo "  and share the tools that shape our world." >> "$OUTPUT"
47 echo "" >> "$OUTPUT"
48 echo "  One day I will build $BUILD and release it" >> "$OUTPUT"
49 echo "  freely – because I have benefited from those" >> "$OUTPUT"
50 echo "  who came before me, and I intend to give back." >> "$OUTPUT"
51 echo "" >> "$OUTPUT"
52 echo "  Open source is not just a license." >> "$OUTPUT"
53 echo "  It is a promise to the world." >> "$OUTPUT"
54 echo "" >> "$OUTPUT"
55 echo "  – $(whoami) | $DATE" >> "$OUTPUT"
56 echo "===== " >> "$OUTPUT"
57
58 echo "  Manifesto saved to $OUTPUT"
59 echo ""
60
61 cat "$OUTPUT"
62
63 echo ""
64 echo "=====
65 echo "  manifesto is ready!"
66 echo "=====
67 echo ""
68

```

OUTPUT:

```
@Priyasinghh28 →/workspaces/oss-audit-24MEI10017 (main) $ bash scripts/script5.sh
Manifesto saved to manifesto_codespace.txt

=====
MY OPEN SOURCE MANIFESTO
Written on: 29 March 2026
=====

(This is based on your answers above)

I believe that technology should serve people
not the other way around.

Every single day, I use git – a tool built
openly and shared freely by people who believed
that knowledge belongs to everyone.

To me, freedom means choice – and that matters.
In software, that is the right to read, change,
and share the tools that shape our world.

One day I will build a study app and release it
freely – because I have benefited from those
who came before me, and I intend to give back.

Open source is not just a license.
It is a promise to the world.

– codespace | 29 March 2026
=====

=====
manifesto is ready!
=====
```

Reference Resources

On open source philosophy

- GNU Project — Free Software Definition
- Open Source Initiative — Open Source Definition
- Eric S. Raymond — *The Cathedral and the Bazaar*

Git and Version Control

- Official Git Documentation — <https://git-scm.com/docs>
- GitHub Guides and Documentation
- Pro Git Book (available online for free)

On Linux and shell scripting

- *The Linux Command Line* by William Shotts
- GNU Bash Manual — <https://www.gnu.org/software/bash/manual>
- Linux man pages (used through terminal commands like `man git`)

For finding license texts

License Information

- SPDX License List — <https://spdx.org/licenses>
- Choose a License — <https://choosealicense.com>
- Git official repository (LICENSE file)

A closing thought

This project helped me understand that open source is not just about free tools, but about collaboration and shared knowledge. While studying Git, I realized how important community contributions are in building reliable software.

Overall, it changed my perspective — from just using software to appreciating the idea of contributing and learning from others

